

SYSTEM DESIGN INTERVIEW

Design a Real-Time Collaborative Editor

Operational Transform, CRDTs & Multi-User Conflict Resolution

~ 35 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Overview & Scale
- 02** · Clarifying Questions
- 03** · Capacity Estimation
- 04** · High-Level System Architecture
- 05** · Core Design Decisions: OT vs CRDT
- 06** · Data Models & Storage
- 07** · Concurrent Edit Processing
- 08** · OT/CRDT Deep Dive
- 09** · Presence & Cursor Tracking
- 10** · Document Load, Save & Offline Mode

[11](#) · Failure Modes & Recovery

[12](#) · Interview Playbook

01

Problem Overview & Scale

What we are building

Design a real-time collaborative document editor similar to **Google Docs**. Multiple users edit the same document simultaneously, see each other's cursors and edits within sub-second latency, and continue working when offline — with all changes merging deterministically when they reconnect.

Scale Targets

- **1 billion documents** stored across all users
- **100 million daily active users** opening, editing, and sharing documents
- **50 concurrent users** editing the same document simultaneously (typical; can spike higher)
- **Sub-second latency** for seeing others' edits (cursor positions, text changes)
- **Offline support:** users can edit without internet; sync when reconnecting
- **Conflict resolution:** concurrent edits from multiple users must merge correctly

Key Challenges

- **Consistency:** ensure final document state is identical across all clients (eventual consistency)
- **Low latency:** optimistic updates on the client for instant UI response
- **Conflict resolution:** when two users insert text at the same position, one must win deterministically
- **Scalability:** support millions of documents, billions of edits, high-throughput revision logs
- **Undo/Redo:** per-user undo must not affect others' edits
- **Offline & sync:** buffer ops locally (IndexedDB), merge on reconnect

02

Clarifying Questions

Pinning down the scope

Scope	Question	Assumption
Scope	Real-time consistency requirement?	Eventual consistency; broadcast within 500ms–1s
Strategy	Conflict resolution approach?	Operational Transform (simpler) or CRDT (better at scale)
Offline	Must support offline editing?	Yes; sync on reconnect with conflict detection
Size	Document size limits?	Up to 50MB per document (typical: <1MB for collaborative editing)
History	Revision history retention?	Last 30 days of full history; snapshots every 100 edits
Users	Max concurrent users per doc?	Design for 50; graceful degradation up to 100+
Presence	Show cursor positions & user list?	Yes; real-time presence, color-coded cursors, typing indicators
Permissions	Access control needed?	Yes; owner, editor, viewer roles; share permissions

03

Capacity Estimation

Math for scale

Concurrent Sessions & Operations

- 100M DAU $\times$ 10% concurrent: **10M users online at peak**
- Avg 3 docs per user session: **~30M active document sessions** at peak
- 50 users per doc (median; 90th percentile 100+): key bottleneck = document session
- Per-user edit rate: 1–5 operations/second (typing, delete, format changes)
- Peak edit throughput: 30M docs $\times$ 2 ops/sec avg = **60M ops/sec** globally

WARNING

Worst-Case, Not Steady-State

The 60M ops/sec figure assumes **every** active session is actively typing at 2 ops/sec at the same instant — a worst-case envelope for capacity planning, not a steady-state load. Real traffic is bursty: most sessions are idle (reading, scrolling, formatting). A more realistic steady-state is 5–10 $\times$ lower. Size the system for the peak; don't pay for it 24/7.

Bandwidth & Latency

- Op size: ~200 bytes per operation (JSON: type, pos, text, user_id, timestamp)
- Broadcast fanout: 50 concurrent users per doc $\rightarrow$ 50 $\times$ 200 bytes = **10KB per op per doc**
- Peak inbound: 60M ops/sec $\times$ 200 bytes = **12 GB/sec** (aggregated across regions)
- Presence updates: 1 cursor/user/100ms = 500 cursors/doc per second at 50 users
- Target latency: optimistic update immediate (0ms); server ack <100ms; peer ops <500ms

Storage & Revision History

- Document snapshots: 1B docs $\times$ 50KB avg = **50PB** (compressed: ~20PB with gzip)
- Revision log: 1 doc may have 10K–100K ops over lifetime; 1B docs $\times$ 50K ops avg $\times$ 200 bytes = **10EB** (archived/pruned)
- Hot storage (last 30 days): ~100M active docs $\times$ 100 ops/day $\times$ 200 bytes = **2PB**
- Index: {doc_id, user_id, timestamp} for revision lookup; sharded by doc_id

KEY

Key Numbers

Peak ops: **60M/sec** (envelope) · Inbound bandwidth: **12 GB/sec** · Snapshot storage: **~20PB** compressed
· Hot tier: **~2PB** for the last 30 days

04

High-Level System Architecture

Three layers: client, real-time, persistence

The system is divided into three main layers — clients, the real-time collaboration tier (WebSocket gateway + OT engine + presence), and persistence (snapshots, op log, archive). Each document is pinned to a single OT engine instance via sticky session so the engine can serialize ops without distributed consensus.

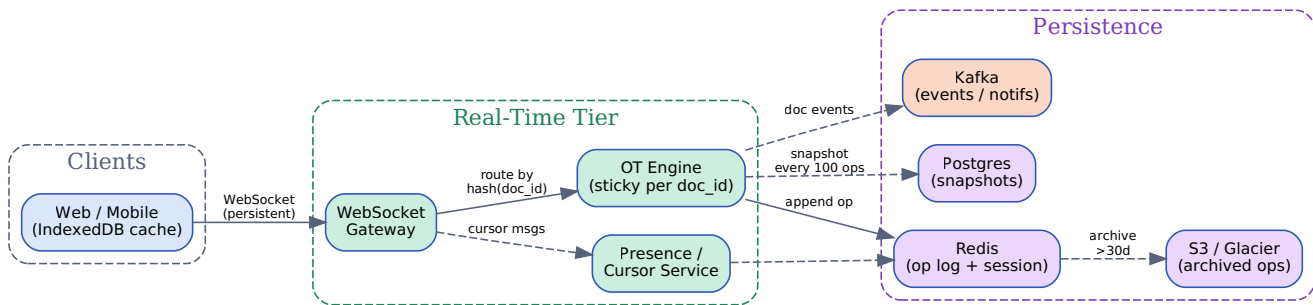


Figure 4.1: Client → WebSocket gateway → OT engine (sticky session per doc) → Postgres snapshots, Redis op log, Kafka events, S3 archive.

Client Layer

- **Web/Mobile clients:** browser or native app; maintains local doc state in memory
- **WebSocket connection:** persistent bidirectional channel to collaboration server
- **Local cache (IndexedDB):** store snapshot + recent ops for offline mode
- **Optimistic updates:** apply edits immediately to UI while sending to server

WebSocket Gateway & Load Balancing

- **Per-document routing:** all users editing doc_id=X route to same server instance (sticky session)
- **Horizontal scaling:** Hash(doc_id) → server instance; scale by sharding docs across backends
- **Connection multiplexing:** one WebSocket per client; multiple docs per session via msg routing
- **Heartbeat & reconnection:** detect stale connections; fast reconnect with op backfill

Collaboration Services

- **OT/CRDT Engine:** transform incoming ops against concurrent ops; compute final state
- **Document Session Manager:** maintain active doc state, session metadata, user list per doc
- **Cursor/Presence Service:** track real-time cursor positions, typing indicators, user colors

Storage Layer

- **Document Store (Postgres / Firestore / Bigtable):** persistent snapshots; schema: {doc_id, snapshot, version, timestamp}
- **Op Log (Redis):** append-only log of recent operations; indexed by {doc_id, version} for quick replay
- **Redis session cache:** hot session state (active user list, last known version, in-flight ops)
- **S3 / Glacier archive:** ops older than 30 days flushed for long-term retention

Background Services (Async)

- **Async Persistence:** periodically save snapshots (every 100 ops or 30 seconds)
- **Spell Check Engine:** real-time spell checking; flag errors without blocking edits
- **Notification Service (Kafka):** broadcast doc changes, @mentions, permission updates

05

Core Design Decisions: OT vs CRDT

The defining trade-off for this system

The choice between **Operational Transform (OT)** and **Conflict-free Replicated Data Types (CRDT)** is the single most consequential decision in this design. It cascades into the data model, the offline story, the server topology, and how undo behaves.

Aspect	Operational Transform (OT)	CRDT
Consistency	Strong eventual consistency; requires central server	Weak eventual consistency; decentralized possible
Transformation	Must compute transform(op_a, op_b); non-commutative	Ops commute; no transform needed
Offline support	Harder; must detect & resolve conflicts on reconnect	Natural; automatic merge via unique IDs
Complexity	Complex xform logic; hard to implement correctly	Simpler to reason about; deterministic
Performance	Fast for small N users; slower with many concurrent edits	Consistent performance; scales well
Undo/Redo	Complex; must track op dependencies	Simpler; each op has unique ID
Production use	Google Docs, Wave (centralized)	Figma, Apple Notes, Automerge, Yjs (local-first)

Recommended Approach: Hybrid (Central OT)

- **Primary:** central server-side OT engine for authoritative conflict resolution
- **Backup:** client-side CRDT-like merge for offline sync and eventual consistency fallback
- **Rationale:** OT + central server = predictable, testable, audit-friendly (Google Docs approach)
- **Scaling:** shard by doc_id to keep OT engine simple per shard; docs don't move between servers

KEY

Design Decision

We will use Operational Transform with a central OT engine and sticky sessions per doc. All edits route to one server; that server transforms ops and broadcasts results. This trades some scalability (can't horizontally scale per doc) for simplicity and correctness. As a rough estimate, with ~10MB of in-memory state per active doc (snapshot + recent op buffer + presence) and ~100 ops/sec per doc to transform and

fan out to 50 peers, a 16-core / 64GB server can plausibly host on the order of a few thousand concurrent docs before memory or fanout CPU saturates; treat this as an envelope, not an SLA.

06

Data Models & Storage

Documents, ops, and session state

Document Entity

```
message Document {
  string    doc_id      = 1; // unique ID (UUID)
  string    title       = 2;
  string    content     = 3; // current full text (UTF-8)
  int64     version     = 4; // Lamport version counter
  Timestamp created_at  = 5;
  Timestamp updated_at  = 6;
  string    owner_user_id = 7;
  repeated Permission permissions = 8; // owner / editor / viewer
}
```

Core document metadata stored in Postgres / Firestore / Bigtable.

Operation Log Entry

```
message Operation {
  string    doc_id      = 1;
  int64     version     = 2; // monotonic version for this doc
  string    user_id     = 3;
  Timestamp timestamp    = 4;
  string    type        = 5; // 'insert' | 'delete' | 'format'
  int32     position    = 6; // char position in document
  string    content     = 7; // inserted text (for insert)
  int32     length      = 8; // chars deleted (for delete)
  map<string,string> attrs = 9; // {bold, italic, ...}
}
```

Immutable operation logged in append-only revision log.

Session & Cursor State (Redis)

```
// Redis key: session:{doc_id}
{
  "version": 12500,           // latest applied op version
  "users": {
    "alice": {user_id, cursor_pos, color, typing: true},
    "bob":   {user_id, cursor_pos, color, typing: false}
  }
}
```

```
},  
"pending_ops": [  
  {version: 12501, op_obj},  
  {version: 12502, op_obj}  
]  
}
```

Volatile session state for active document editing.

Snapshot Storage Strategy

- **Periodic snapshots:** every 100 operations or 30 seconds; store {doc_id, snapshot_version, content, timestamp}
- **Snapshot + log:** on load, fetch latest snapshot, then replay ops from that version forward
- **Log cleanup:** archive ops older than 30 days; keep only last 3 snapshots per doc to save space
- **Compression:** store snapshots as gzipped JSON; 50KB avg → 5KB compressed

07

Concurrent Edit Processing

How OT actually transforms two edits

The heart of the system: how concurrent edits from multiple users are transformed and merged into a convergent state. The transform diagram below shows the canonical case — two users insert at overlapping positions; the server transforms one against the other so all clients end at the same final string.

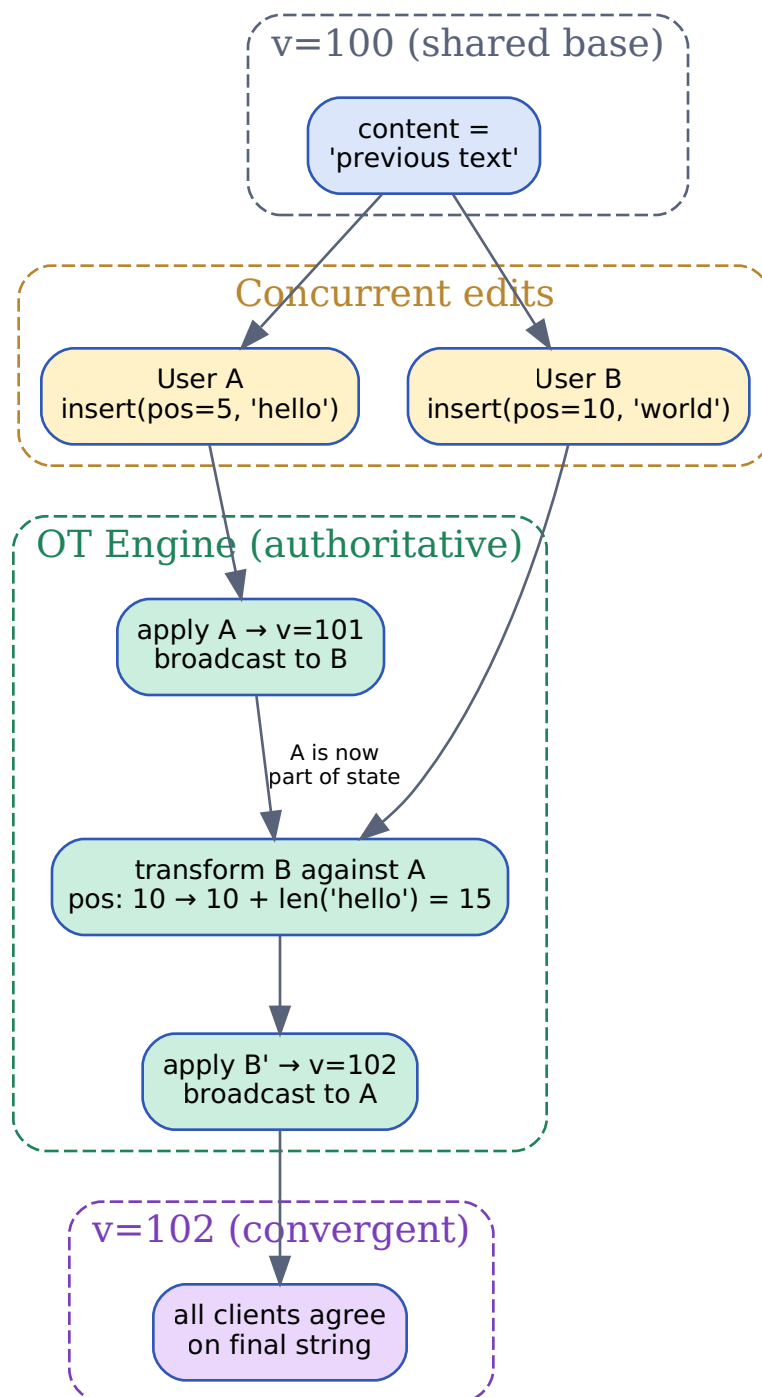


Figure 7.1: Two concurrent inserts at $v=100 \rightarrow$ server transforms B's position past A's insertion $\rightarrow$ both clients converge at $v=102$.

The OT Transformation Algorithm (Simplified)

- **Scenario:** User A inserts 'hello' at position 5; User B inserts 'world' at position 10. Both arrive at server.
- **Server state before either op:** version=100, content='previous text'

- **Op A arrives:** `insert(pos=5, 'hello')`, applied → version=101, broadcast to B
- **Op B arrives:** `insert(pos=10, 'world')`, but conflicts with A's position shift
- **Transform step:** adjust B's position against A: $\text{pos} = 10 + \text{len}('hello') = 15$ (because A's insertion shifts everything after)
- **Apply transformed B:** `insert(pos=15, 'world')`, applied → version=102
- **Broadcast:** both A and B receive transformed ops; convergence guaranteed

Why This Works (Convergence Proof Sketch)

- **Commutativity:** if the transform function is commutative, final state is independent of op order
- **Idempotency:** a client that sent op_i won't re-apply it when broadcast back (dedup by op_id)
- **Causality:** each op includes a version number; ops must be applied in version order

INSIGHT

OT Transformation Invariant

The key invariant: $\text{transform}(\text{op_a}, \text{op_b}) \equiv \text{transform}(\text{op_b}, \text{op_a})$ (up to reordering). This ensures all clients converge to the same final state even if they apply ops in different orders. The server is authoritative; it decides version order.

08

OT/CRDT Deep Dive

Why production OT is harder than it looks

Operational Transform (OT) Essentials

- **Core idea:** given two concurrent ops, compute a third op that applies both edits correctly
- **Transform function:** $op' = T(op_a, op_b)$ where op' is op_a adjusted for op_b 's effects
- **For insert operations:** if op_b inserts before op_a 's position, shift op_a 's position forward
- **For delete operations:** delete positions shift based on prior insertions; overlapping deletes handled carefully
- **String vs tree:** most collaborative editors use string (linear sequence); complex docs use tree (nodes, hierarchy)

WARNING

Why This Is Hard in Practice

Correct OT requires the transform function to satisfy **TP1** (convergence for two concurrent ops) and **TP2** (convergence for three or more concurrent ops applied in any order). TP2 is famously difficult to satisfy — multiple published OT algorithms were later shown to violate it under specific edit sequences. OT must also preserve user *intent* (e.g., not splitting a word a user just typed), which adds further case analysis. These pitfalls are why most modern collaborative systems (**Figma**, **Automerger**, **Yjs**) chose CRDTs instead, trading larger payloads for a merge function that is correct by construction.

CRDT (Conflict-free Replicated Data Type) Alternative

- **Core idea:** assign unique IDs to each character; no transform needed because order is via ID comparison
- **Example — Yjs CRDT:** each char gets {unique_client_id, sequence_num}; merge sorts by ID
- **Advantage:** works offline; automatic eventual consistency; simpler merge logic
- **Disadvantage:** higher memory overhead (IDs per char); harder to implement undo/redo per-user
- **Best for:** local-first apps (Figma, Apple Notes); eventual consistency acceptable

Practical Trade-Offs

- **OT:** smaller payloads, faster (no metadata per char), audit-friendly (linear history); needs central server
- **CRDT:** scales offline editing, supports true P2P, deterministic merge; larger payloads, complex internal data structures
- **Hybrid approach:** use OT on the server for consistency; clients cache CRDT-style IDs for offline merge

09

Presence & Cursor Tracking

Showing who's there and where

Real-Time Cursor Positions

- **Update frequency:** send cursor position every 100ms (or on significant movement) to avoid flooding
- **Payload:** {user_id, cursor_pos, selection_start, selection_end, color} ~50 bytes
- **Broadcast:** server multicasts to all other users in the session (fanout to N-1 clients)
- **Latency:** target <200ms for cursor visibility

Presence Heartbeat & Timeout

- **Heartbeat:** client sends keepalive every 30 seconds; includes user status (online, away, typing)
- **Server timeout:** remove user from active list if no heartbeat for 2 minutes (network stale)
- **Graceful exit:** user closes editor → send 'user left' event → remove from all clients' presence list

User Colors & Avatars

- **Color assignment:** on session join, server picks next color from palette (8–10 colors, rotate through users)
- **Avatar display:** show user initials + color in sidebar and next to cursor
- **Consistency:** same user always same color across all docs and sessions

10

Document Load, Save & Offline Mode

The read/persistence path and offline sync strategy

Loading a Document

1. User opens doc → fetch latest snapshot from Postgres/Firestore (indexed by doc_id)
2. Query revision log: get all ops since snapshot version
3. Replay ops on snapshot in version order → current document state
4. Initialize WebSocket session; subscribe to real-time ops for this doc
5. Render UI with full document + user list

Periodic Snapshot & Save

- **Trigger:** every 100 operations OR 30 seconds, whichever comes first
- **Process:** server saves {doc_id, snapshot_content, version, timestamp} to Postgres
- **Log cleanup:** ops older than snapshot version can be archived (keep 30 days for audit)
- **Latency:** async; doesn't block real-time editing

Offline Mode & Reconnection

- **On disconnect:** client detects WebSocket close; stop sending ops to server
- **Local buffering:** cache all edits to IndexedDB (browser's local database)
- **Optimistic UI:** continue showing edits in UI immediately (local only)
- **Reconnection detect:** periodically try to reconnect (exponential backoff)
- **Sync on reconnect:** send buffered ops to server; server transforms against concurrent edits during offline period
- **Conflict resolution:** if conflict detected, show user merge UI or accept server version

TIP

Offline Design

Offline is supported via local IndexedDB cache + buffered ops. When reconnecting, send buffered ops with version info. Server compares against its state; if diverged, apply transformation or ask client to resolve. For most use cases (single user editing offline), automatic merge succeeds.

11

Failure Modes & Recovery

What can go wrong and how we recover

Failure Mode	Impact	Recovery Strategy
WebSocket disconnect	Ops don't reach server; client unaware	Client detects close event; buffer ops; exponential backoff reconnect
Server crashes	All active sessions lost; pending ops may be lost	Failover to hot standby (Redis backup); clients reconnect; re-sync from snapshot + log
Op loss during persist	Ops applied to docs but not persisted	Periodic snapshots are synchronous/durable; ops logged to journal before ACK
Split-brain (network partition)	Clients diverge; can't reach server	Clients buffer ops locally; on reconnect, server version wins; client merges or discards local ops
Concurrent op collision	Same position edited by 2+ users simultaneously	OT transform resolves; server version order is authoritative
Slow client (lagging)	Client receives ops out of order or with gaps	Client queues ops by version; applies once version is contiguous

WARNING

Graceful Degradation

Prioritise the editing path: if presence, spell check, or notifications fail, the document must still be editable. Always serve the latest snapshot from cache; degrade real-time fanout before degrading the OT engine itself.

12

Interview Playbook

How to present this in 45 minutes

Structure for a 45-minute interview on real-time collaborative editing. Lead with the OT-vs-CRDT trade-off — that's the test of whether you understand the problem.

Time	Topic	What to Cover	Depth
0–5 min	Problem statement	1B docs, 100M DAU, 50 users/doc, sub-second latency	Clarifying questions
5–10 min	Requirements	Functional (edit, share, history) & non-functional (99.9% availability, <500ms op latency)	Trade-off discussion
10–15 min	Capacity estimation	60M ops/sec envelope, 12 GB/sec inbound, 50PB snapshots, 2PB hot storage	Back-of-envelope math
15–25 min	High-level architecture	WebSocket gateway, OT engine, doc store, Redis session cache	Draw both diagrams
25–30 min	OT vs CRDT	Compare approaches; justify OT for this use case (central + sticky sessions); mention TP1/TP2	Design decision
30–35 min	Deep dive: one topic	OT algorithm & transform, or presence/cursors, or offline sync	Interviewer picks
35–40 min	Failure scenarios	Handle WebSocket drops, server crashes, op loss, merge conflicts	Resilience discussion
40–45 min	Wrap-up & trade-offs	Scalability limits (per-doc shard), latency (network RTT), cost (storage)	Summary & questions

Key Points to Emphasize

- **Scalability:** shard by doc_id (sticky sessions); each shard handles a few thousand docs (envelope, not SLA)
- **Consistency:** strong eventual consistency via OT; server authoritative
- **Latency:** optimistic updates (0ms), server ack (<100ms), broadcast (<500ms)
- **Reliability:** async persistence; hot failover; offline sync on reconnect
- **Trade-offs:** OT is conceptually simpler than CRDT but production-correct OT is hard (TP2, intent preservation) — that's why Figma / Automerge / Yjs picked CRDTs

KEY

Key Takeaways

Real-time collaborative editing hinges on **Operational Transform** for conflict resolution and **central server authority** for consistency. Use sticky sessions per doc to avoid distributed consensus. Store snapshots + op logs for durability and recovery. Support offline editing via local cache and merge on reconnect. Design for 50 concurrent users per doc with sub-500ms op propagation latency — and remember that modern systems (Figma, Automerge, Yjs) chose CRDTs precisely because production-grade OT is hard to get right.

Further Reading & References

- Google Wave / Docs architecture: [USENIX OSDI '10 — Real-Time Collaborative Editing](#)
- Operational Transform (Ellis & Gibbs, 1989): foundational OT paper
- Yjs library: github.com/yjs/yjs — production CRDT for collaborative editing
- Automerge: automerge.org — JSON-like CRDT with rich text support
- Figma multiplayer tech: figma.com/blog/how-figmas-multiplayer-technology-works/